Lab Experiment 5- Designing a Pipelined RISC Processor (Part 2)

Objective

The objective of this lab is for you to build the building blocks required to construct a pipelined RISC (Reduced Instruction Set Computer) processor capable of executing 16 different instructions including load, store, arithmetic and logical operations. The processor will not perform branch and control instructions. You will design and implement the functional units of the processor, including the instruction unit, instruction decode, instruction execution, and register file.

In previous lab:

- You completed the given templates for each component: instruction unit, decode unit, register file and execution unit
- You simulated the behavior of each of the individual units.
- You learned the instruction formats and ALU operations.

In this lab:

- You will complete the processor design by adding an instruction memory and data memory, integrating all the components, and performing comprehensive testing of the processor.
- The instruction and data memory are given to you. Your main task is connecting all the components and simulate and analyze the entire system.

Lab Equipment and Software

Computer with ModelSim software installed.
Text editor or Verilog Integrated Development Environment (IDE)

Pre-lab Preparation

1. Before coming to the lab, have all your designed building blocks from the previous lab ready and read this manual.

Overall System Specifications

- 4-stage pipelined RISC processor
- Instruction set consisting of 16 instructions (including load, store, NOP, arithmetic, logical, and shift instructions)
- Register file with 8 general-purpose registers.
- ALU (Arithmetic Logic Unit) for executing arithmetic, logic, and shift operations.
- Assume instruction memory has only 32 locations.
- Assume data memory has only 16 locations.
- Use the given code uploaded on Brightspace for data and instruction memory

Functional Description for each unit

Previously you designed the following units:

- **Instruction Unit:** The instruction unit is responsible for fetching instructions from memory and managing the program counter (PC).
- **Instruction Decode:** The instruction decode unit is responsible for decoding each instruction by extracting various parts from the instruction, such as registers and immediate values.
- **Instruction Execution:** The instruction execution unit performs the actual operation specified by the instruction, involving the ALU and other necessary components.
- **Register File:** The register file is responsible for providing the required registers for each operation and updating the destination register.

Task 1: Structural Modelling of the Processor

In this task, your objective is to interconnect all submodules to assemble the processor. Submodules are given in Figure 1. Pay close attention to the input and output signals for each unit, ensuring you understand their origins and destinations. For example, the decode unit extracts information about the source and destination registers (*opnda, opndb, dst*) and these information must be provided to the register file in order to select which source registers we are selecting and in which register we are writing. For example, ***opnda*** output from the decode unit must be connected to ***opnds_addr*** input of the register file. Keep in mind that you need to use wires in your top-level design to connect the inputs and outputs from different modules.

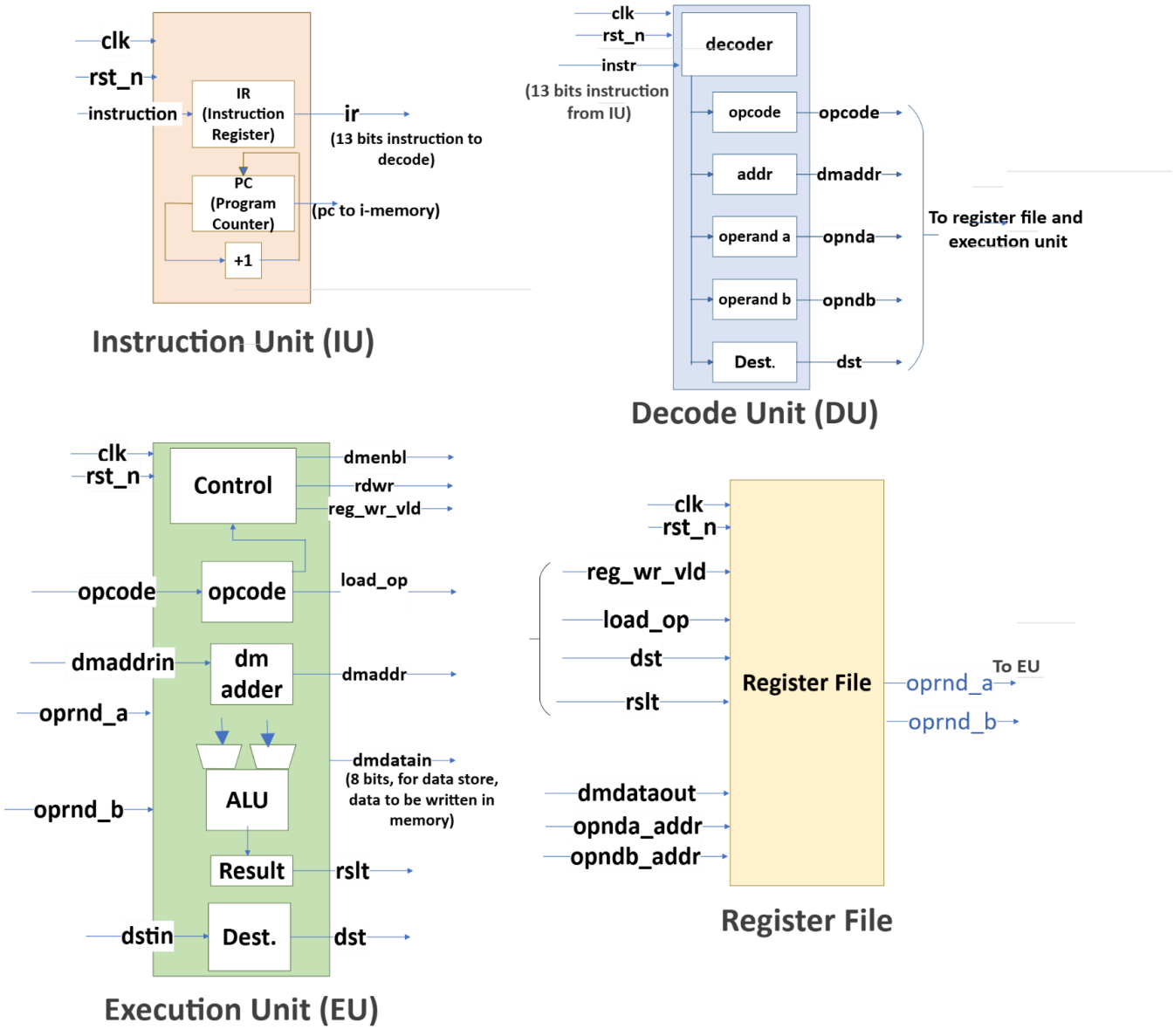

Figure 1_design units for the RISC processor

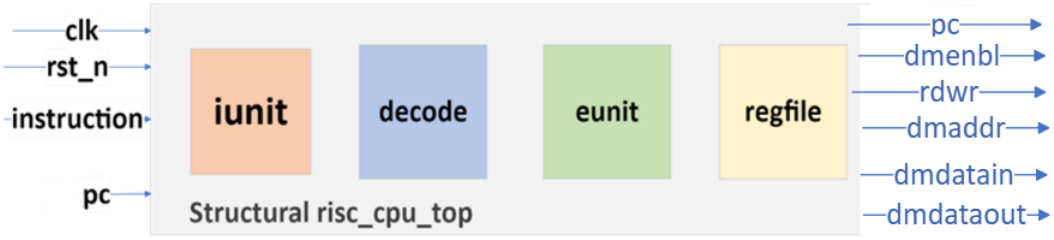

Figure 2_Abstract top-level block diagram for CPU

The abstract top-level design is provided in Figure 2, and you can refer to the template available on Brightspace for guidance. You do not need to test this module. You see the systems outputs some signals related to data memory including read/write signal (*rdwr*), data memory enables (*dmenbl*), address (*dmaddress*), data to be read from the memory (*dmdatain*), data to be stored in the memory (*dmdataout*). These signals must be connected to data memory in the next task.

Task 2: Structural Modelling of the System Including Data and Instruction Memory

For this task, you will be connecting the data memory and instruction memory to the CPU system that you built in task 1. The block diagram is illustrated in Figure 3.

In practice, programs are typically compiled into machine code by a compiler and loaded into the system's memory using a loader. These tools are often integrated into an integrated development (ID) software package. Developing a compiler is beyond the scope of this course. Therefore, in this case, we have manually assembled the program and will write the machine code and essential data into the instruction and data memories. You can utilize the provided template on Brightspace for this purpose. The Verilog code for data and instruction memory are also given on Brightspace.

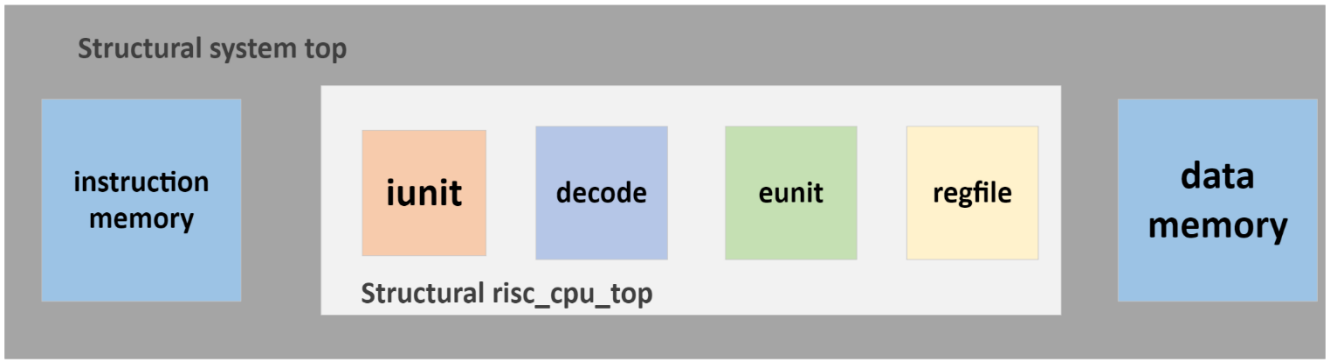

Figure 3_Top-level diagram of system

```
1
2  ; Load values from memory into registers
3  ld   r0, mem[0000]
4  ld   r1, mem[0001]
5  ld   r2, mem[0010]
6  ld   r3, mem[0011]
7  ld   r4, mem[0100]
8  ld   r5, mem[0101]
9  ld   r6, mem[0110]
10 ld   r7, mem[0111]
11
12 ; Perform arithmetic and logic operations
13 add  r0, r0, r1        ; r0 = r0 + r1
14 sub  r1, r7, r6        ; r1 = r7 - r6
15 and  r2, r2, 25        ; r2 = r2 AND 25
16 or   r3, r3, r4        ; r3 = r3 OR r4
17 xor  r4, r4, r4        ; r4 = r4 XOR r4 (clears r4)
18 inc  r5, r5, r0        ; r5 = r5 + r0
19 dec  r6                ; r6 = r6 - 1
20 not  r7                ; r7 = NOT r7
21 neg  r0                ; r0 = -r0
22 shr  r1                ; r1 = r1 >> 1 (logical shift right)
23 shl  r2                ; r2 = r2 << 1 (logical shift left)
24 ror  r3                ; r3 = rotate right
25 rol  r4                ; r4 = rotate left
26
27 ; Store values from registers back to memory
28 ; in locations 1000 to 1111
29 sd   r0, mem[1000]
30 sd   r1, mem[1001]
31 sd   r2, mem[1010]
32 sd   r3, mem[1011]
33 sd   r4, mem[1100]
34 sd   r5, mem[1101]
35 sd   r6, mem[1110]
36 sd   r7, mem[1111]
```

Figure 4_test program

Take screenshots of the simulations, make annotations to make the waveforms more readable. Use hexadecimal radix for the content of the registers, opcode, operands, pc and result.

Task 3: Testbench Design and Simulations

Testbench is given on Brightspace to test the system. We will run the program shown in figure 4, on the processor. In this program, first we load data stored in 8 consecutive locations in data memory into the registers in register file. Then we perform operations on the registers and store the results in the memory again. This program is hand-assembled using the instructions format and opcodes given in lab 4 document. Test the entire system and make screenshots of the waveform and explain the processor operation.

Lab Report

Write a lab report documenting your work on the design units implementation. Your report should include the following sections:

1. **Introduction**: Provide a brief introduction to the lab objective and the concepts covered.
2. **Task 1:** take a screenshot of successful compilation of the CPU system.
3. **Task 2:**

- Take a screenshot of the successful compilation of the system.
- Include screenshots of the content of registers after all loads, and after the execution of the arithmetic and logical instructions.

4. **Conclusion**: Summarize your experience and key takeaways (e.g. any challenges, solutions) from the lab. Reflect on what you have learned in this experiment.
5. Include and submit the complete Verilog codes for your module and the testbench **as a separate file on Brightspace**.

Reference:

Cavanagh, Joseph. *VeriLog HDL: digital design and modeling*. CRC press, 2017.